Intermediate Progress Report: CineMatch

Hybrid movie recommendation system with cold-start support

CS4210 – Machine Learning and Its Applications
Project type: Applied project

Project idea

I am building **CineMatch**, an end-to-end movie recommendation prototype that returns ranked movie suggestions after either selecting an existing MovieLens user or entering a few seed ratings for a new user. The core machine learning goal is still the same as in the proposal: combine collaborative filtering with content features so the system remains useful when interaction data is sparse or when some items are effectively cold start. At the intermediate stage, the project has moved beyond the idea stage into a working prototype with training code, offline evaluation, learned embeddings, and a seeded recommendation demo.

Current methods / pipeline

The current prototype uses the MovieLens 100K dataset: explicit ratings from 943 users on 1682 movies, plus metadata from `u.item` (title, release year, and 19 genre flags). The current system has five stages:

- **Data layer:** parse ratings and metadata, build user/item indices, and create evaluation splits.
- **Warm-start baselines:** a popularity recommender (Bayesian item mean), a neighborhood collaborative-filtering baseline, and an explicit matrix-factorization model with user/item bias terms trained by SGD.
- **Cold-start module:** remove a set of items entirely from collaborative training, then learn a ridge-regression mapper from metadata to the item latent factors learned on warm items.
- **Evaluation layer:** report RMSE and MAE for rating prediction, plus Precision@10, Recall@10, NDCG@10, and catalog coverage for top-N recommendation.
- **Inference / demo layer:** estimate a new user's latent vector from 5–10 seed ratings and generate top recommendations with short explanations based on nearest seed item and shared genres.

For warm-start testing I currently use the official `u1.base/u1.test` split. For cold-start testing I created a custom item holdout: 160 sufficiently rated movies are removed from collaborative training, and ranking is evaluated only on those held-out items.

Implementation progress

I implemented the current prototype from scratch in Node/JavaScript so I could inspect every stage of the pipeline. The working code already includes: data loading and metadata parsing; train/test split handling; a popularity baseline; a neighborhood CF baseline; matrix-factorization training with per-epoch tracking; ranking and error-metric evaluation; a cold-item holdout generator; metadata-to-latent regression; and a seeded recommendation mode that writes example outputs and images for the demo. The current code is still experimental rather than polished: most of the logic is in one working script that trains models, exports tables/JSON, and renders demo assets. Refactoring the code into cleaner modules is one of the final-phase tasks.

Current results / prototype status

Table 1 shows the current warm-start results on `u1.test`. The most important result is that matrix factorization now gives the best rating-prediction accuracy, reaching **RMSE 0.9539**, while the popularity baseline is much worse on RMSE. However, ranking quality is not yet aligned with RMSE: on this split, the simple popularity baseline still has the strongest Precision@10 and NDCG@10, so the current MF objective behaves more like a rating predictor than a top-N ranker.

Table 1: Preliminary warm-start results on `u1.base/u1.test`.

Model	RMSE	MAE	P@10	R@10	NDCG@10	Coverage
Popularity	1.0540	0.8478	0.1511	0.0593	0.1482	0.0232
Neighborhood CF	0.9563	0.7477	0.0145	0.0054	0.0167	0.1617
Matrix factorization	**0.9539**	0.7543	0.1173	0.0443	0.1152	0.0285

Training is real rather than hypothetical: on the current split, MF test RMSE decreases monotonically from about 0.994 at epoch 1 to 0.954 by epoch 16. The current seeded demo also works end-to-end. For a new user who likes *Toy Story*, *Twelve Monkeys*, *Star Wars*, *Fargo*, and *Contact*, the prototype recommends titles such as *A Close Shave*, *Schindler's List*, *The Shawshank Redemption*, *The Wrong Trousers*, and *Casablanca*. Figure 1 shows the current product-style mockup generated from the live model outputs.

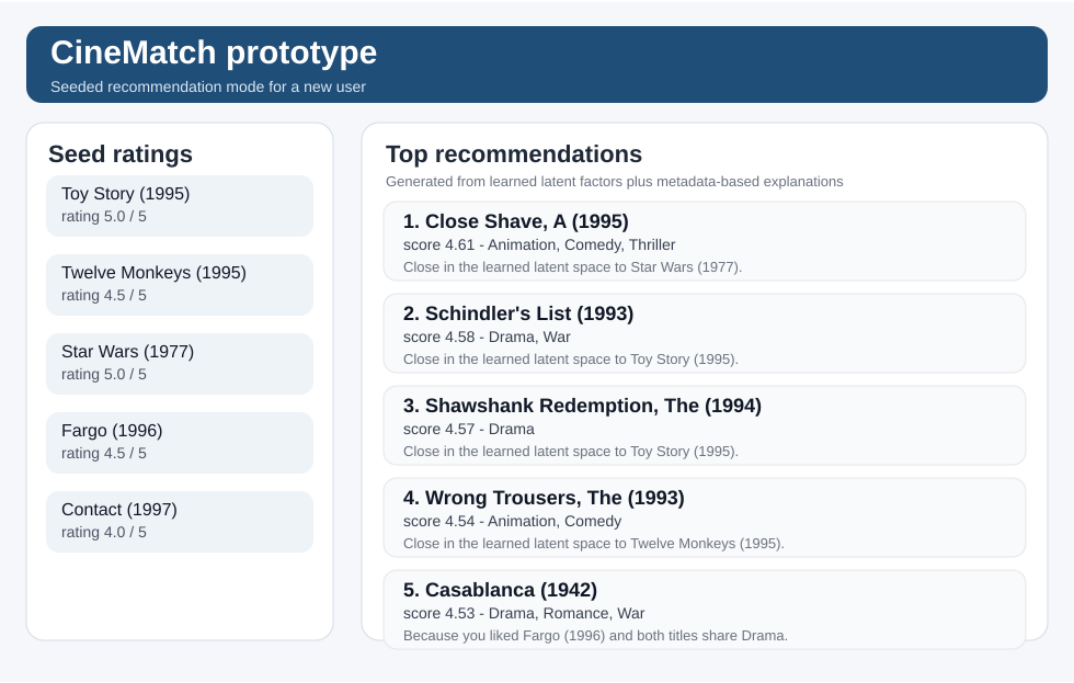

Figure 1: Current prototype status: seeded recommendation mode with generated explanations.

For cold start, I evaluated two metadata-only options on the held-out item split: a simple genre-profile baseline and the current metadata-to-latent hybrid mapper. Table 2 shows that this part is *not finished*: the genre-profile baseline currently outperforms the learned hybrid on top-N ranking across 923 evaluable users.

Table 2: Preliminary cold-start ranking on the held-out item split.

Model	P@10	R@10	NDCG@10	Eval. users
Genre-profile baseline	**0.1161**	**0.1089**	**0.1305**	923
Metadata → latent hybrid	0.0757	0.0470	0.0843	923

Challenges and changes from the original proposal

The most important change is methodological. My proposal originally leaned toward using a library-first hybrid model such as LightFM. For intermediate progress, I instead built a from-scratch explicit MF pipeline plus a metadata-to-latent mapper. This gave me a working end-to-end prototype quickly and made the failure modes easier to inspect, but it also exposed a key issue: optimizing explicit RMSE does not automatically yield strong top-N ranking or cold-start performance.

The second challenge is the cold-start result itself. The current hybrid mapper does *not* yet beat the simpler genre-profile baseline. That is actually useful information: it suggests that reconstructing collaborative item factors from sparse metadata is harder than expected, and it means the final system should either use richer features (for example title tokens/tags) or combine the metadata score with a ranking-oriented collaborative model instead of relying on factor reconstruction alone.

The third issue is software polish. The prototype is functional, but the code needs cleanup and the demo interface is still a generated mockup rather than a finished interactive web app.

Next steps before final submission

Before the final submission, I plan to finish four things. First, I will switch the main recommender toward a ranking-oriented objective or add a re-ranking stage so the personalized model is optimized for Precision@K/NDCG rather than only RMSE. Second, I will strengthen the cold-start side by adding richer content features and testing a blended scorer that combines genre similarity with learned latent predictions. Third, I will refactor the current experimental training script into separate training, evaluation, and demo modules and wrap the inference stage in a small interactive UI. Fourth, I will run a broader evaluation over multiple splits, expand the explanation layer, and prepare the final technical presentation and live demo.

Overall, the project now has meaningful ML implementation, preliminary experiments, a working recommendation prototype, and clear evidence about what currently works and what still needs improvement.